Smart Complaint Management System

Software Architecture and Design Specification

Section 3 Architecture | Member 3 scope
Prepared by Mohammed Zahoor Mashahir | PES1UG24CS579
Team 8 | Version 0.1

3 Architecture

We select a layered modular monolith for SCMS. A single Flask application coordinates complaint workflows through separate authentication, complaint, routing, audit, feedback and query modules. SQLite provides durable storage. This structure supports the fixed university escalation hierarchy while keeping deployment and transaction management manageable for the mini-project.

This architecture provides the component boundaries and data ownership for the Section 4 design. The design supports Students, Faculty-in-Charge at L1, Grievance Committee at L2 and Dean at L3. It uses manual SLA review and in-app notifications. No university ERP, SSO, external email service or background SLA scheduler is required.

3 1 Architecture pattern and rationale

Presentation routes validate requests and invoke application services. ComplaintService coordinates use cases; RoutingPolicy owns level transitions; the Repository isolates SQLite access. Services share an explicit transaction context when a complaint change requires audit, assignment or notification records. The modules are logical components within one deployment unit, not independent network services.

Option	Fit for SCMS	Tradeoff
Layered modular monolith	Selected. Matches Flask and SQLite constraints and allows one atomic workflow transaction.	Modules can be maintained independently, but share deployment and database availability.
Microservices	Separate deployment and service-owned stores would add complexity for a small team.	Distributed writes, authentication and operation are disproportionate to this scope.
Unstructured single module	Simple initially, but duplicates role checks and routing rules.	Does not meet NFR-MAIN-01 separation of functionality.

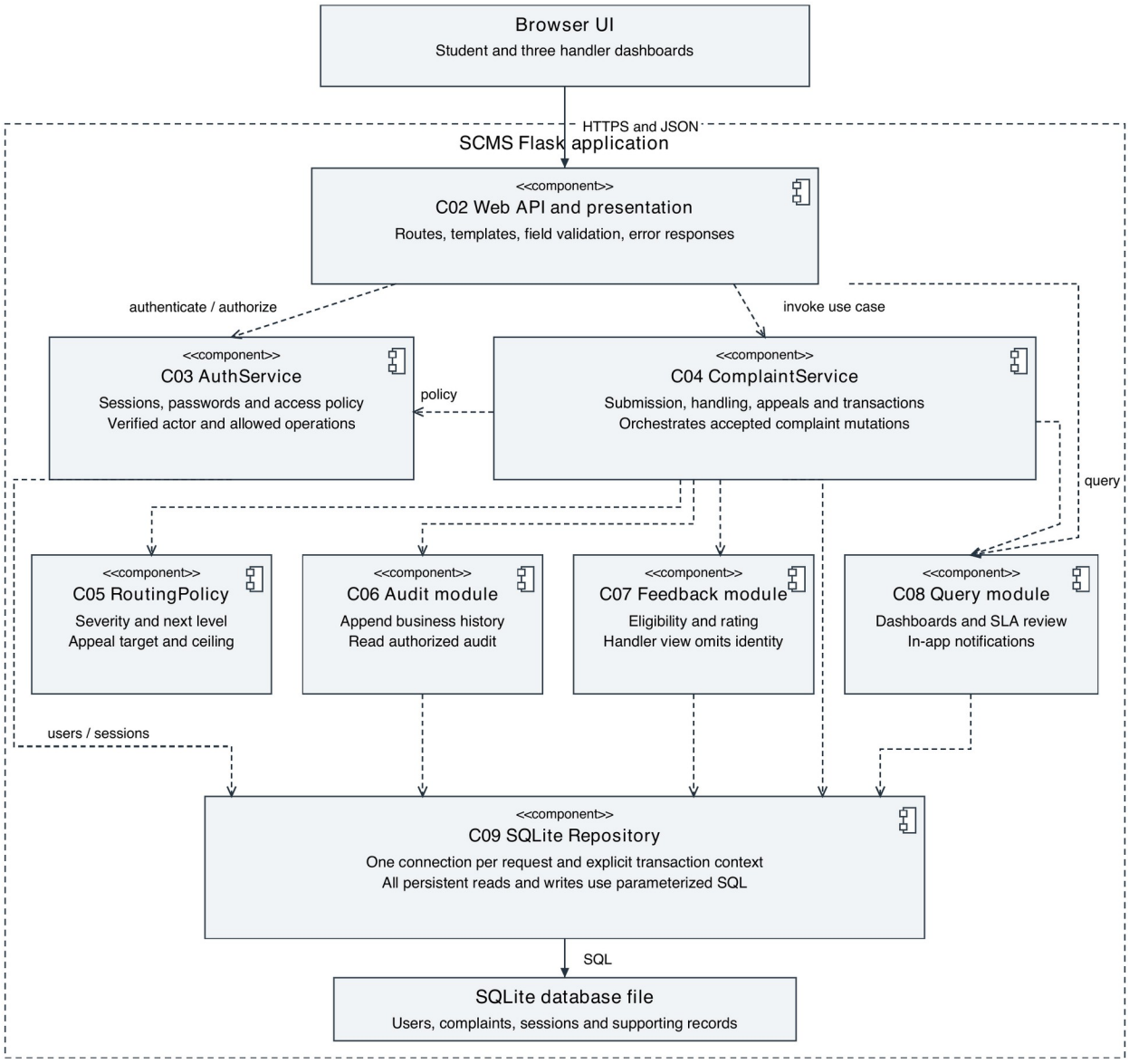
3 1 1 Technology and scope constraints

Python 3, Flask and SQLite are SRS constraints. Use a modern browser and a Windows or Linux host. Dependency versions are pinned when implementation begins. This specification describes proposed architecture; it does not report an implemented web application or passed acceptance tests.

3 2 UML component diagram

Figure AC01 shows the presentation, application and persistence layers. Dependencies point toward the required component. AuthService supplies verified identity and policy; application services enforce it on every protected use case.

AC01 SCMS UML Component Diagram



Dashed arrows show dependencies; solid arrows show external communication or SQL access.
C01 is the Browser UI. RoutingPolicy is pure and has no database dependency.

AC01. C01 is the Browser UI. C02 exposes the browser-facing interface. C04 owns workflow transactions. C06 to C08 access C09 using the same transaction when called during a mutation; read requests use their own request connection.

3 3 Component descriptions and interfaces

ID and component	Responsibility	Communicates with
C01 Browser UI	Registration, login, complaint forms, status and role-specific dashboards. Displays errors and permitted actions.	C02 over HTTPS; never C09 or the database.
C02 Web API and presentation	Flask routes and templates; request parsing, validation, authentication guard and consistent responses.	C03 identity; C04 use cases; C07 feedback; C08 queries.
C03 AuthService	Registration, password verification, session lifecycle and ownership/role/assignment access policy.	C09 users/sessions; verified actor supplied to C02 and service callers.
C04 ComplaintService	Coordinates submission, resolve, reject, escalation and appeal. Owns the transaction boundary.	C03 policy, C05 routing, C06 audit, C08 notices and C09 state.
C05 RoutingPolicy	Pure initialLevel, nextLevel and appealTarget decisions. Enforces L3 ceiling centrally.	C04 only; no persistent state.
C06 Audit module	Appends accepted business events and returns authorized chronological history.	C03 visibility; C09 audit and assignment history.
C07 Feedback module	Resolved-owner eligibility, rating validation, one feedback record and identity-suppressed handler view.	C03 policy; C09 complaints and feedback.
C08 Query module	Filtered dashboards, manual SLA calculation and notifications. SLA escalation delegates to C04.	C03 policy; C04 escalation; C09 read models/notices.
C09 SQLite Repository	Parameterized persistence, request connections, constraints and transaction support.	SQLite file; C03, C04, C06, C07 and C08.

3 3 1 Interface ownership

AuthService, ComplaintService, RoutingPolicy and SQLite Repository retain the interface names in Section 4. Audit and feedback are dedicated modules beneath those interfaces. Query methods support listVisible, audit reads, SLA review and notification reads. Every service receives a verified actor and checks resource policy; no caller treats UI visibility as authorization.

Repository mutation helpers accept the caller transaction. Audit and notification helpers do not open a second independent transaction during a complaint change. This avoids a complaint committing without its required history. Detailed HTTP payloads and error contracts remain in Section 4.

3 4 Runtime workflows and state rules

3 4 1 Submission and assignment

C02 authenticates the Student and validates category, description and severity. C04 asks C05 for the initial level: Low/Medium to L1 and High to L2. C09 inserts the unique complaint, timestamps and initial assignment; C06 records SUBMIT/ASSIGN; C08 creates the role-queue notice. A single successful commit precedes confirmation. Invalid input or failed persistence leaves no partial complaint or success message.

3 4 2 Handler actions and appeal

C03 checks the current assignment before normal handler mutations. C04 validates state, mandatory remarks/reason and expected_version. C05 permits L1 to L2 and L2 to L3; L3 has no next level. A rejection records rejected_level. An owning Student may appeal once while Rejected: L1 to L2, L2 to L3, or L3 final review at L3. A unique appeal constraint and conditional update protect against concurrent duplicate appeals.

Operation	Resulting state	Required invariant
Submit	Submitted at L1 or L2	Student owner, valid fields, persisted unique ID.
Resolve	Resolved	Current handler and nonblank remarks.
Reject	Rejected	Current handler, reason and rejecting level recorded.
Escalate	Escalated at next level	Current handler; L3 ceiling; no terminal-state action.
Appeal	Under Appeal Review	Owning Student; Rejected; not previously appealed.

3 4 3 Dashboards audit and feedback

Students see their own complaints. Faculty/Committee see their current queue. The Dean sees complaints that reached L3 and excludes earlier-level resolutions. Owners and handler levels with current or historic assignment can read audit history; historic visibility grants no mutation right. C07 prompts the owner for feedback on a resolved complaint without existing feedback and omits identity from the handler projection.

3 4 4 Manual SLA review

C08 computes elapsed time from status_updated_at using the configured threshold, default 7 days. A protected overdue view shows the current queue, days overdue and escalation availability. Escalation delegates to C04 and C05 after a fresh permission, state, version and threshold check. The scope of this exception to normal assigned-handler authority requires team review; it is not a general cross-queue handling permission.